Пројекат из предмета Напредни микропроцесорски системи
Лазар Минчић, E1 35/2024

Проширење скупа инструкција факултетског RISC-V 32I процесора уз имплементацију јединице за предвиђање програмских скокова

Увод

Скуп инструкција факултетског процесора проширен је тако да садржи све инструкције bare-metal RV32I архитектуре. Као путоказ коришћена је књига [1]. Постојећем процесору додата је јединица за предвиђање програмских скокова у циљу решавања управљачких хазарда који се јављају приликом извршавања произвољног програма на процесору. Функционалност система верификована је покретањем неколицине тест програма и анализом таласних облика симулације. Резултати су сматрани тачним уколико се поклапају са резултатима из Ripes симулатора [2].

Полазна архитектура процесора

Факултетски процесор има подршку за инструкције сабирања, add; одузимања, sub; битског или, or; битског и, and; сабирања са константом, addi; логичког или са константом, ori; логичког и са константом, andi; учитавања у регистар из меморије, lw; уписа у меморију из регистра, sw; и условни скок уколико су операнди једнаки, beq.

Почетни процесор предвиђа да програмски скок никада неће бити извршен. Ово се зове још и статички предиктор. Предвиђање се врши у IF фази. Уколико се у ID фази утврди да је до скока дошло, if / id регистар се чисти а програмском бројачу прослеђује се адреса скока. Казна за неправилно предвиђен програмски скок износи један тактни циклус. Имајући у виду да се у програмима опште намене већина програмских скокова заправо извршава, ово решење није довољно ефикасно у општем случају.

Проширење сета инструкција

Успешно су имплементиране инструкције померања: sll, srl, sra, slli, srli и srai, остали условни скокови осим beq: bne, blt, bge, bltu и bgeu, инструкције упоређивања slt, sltu, slti и sltiu, инструкције битског ексклузивно или, xor и xori, инструкције U-типа lui и auipc и инструкције безусловних скокова jal и jalr.

sll инструкција помера вредност rs1 за rs2 места у лево и складишти је у rs. ALU јединица измењена је тако да подржи операцију померања у лево. По спецификацији, за померање користе се само најнижих 5 битова rs2. Контролна логика бира одговарајући резултат из datapath-а на основу поља funct3 и funct7, као и вредности сигнала alu_2bit_op. Излаз (за читање) из регистарске банке користи се као улаз у ALU јединицу (осим уколико није дошло до прослеђивања једног или више операнда). Нема приступа меморији за податке, док се у WB фази израчунати податак уписује у одредишни rd регистар. Аналогно се понашају и остале инструкције померања, с тим што је операција коју ALU рачуна другачија (контролни сигнал alu_op_i се разликује). Што се тиче I типа инструкција померања,

контролна логика бира излаз из immediate блока као улаз у ALU уколико је декодована инструкција баш овог типа. Хардвер користи искључиво најнижих 5 бита immediate поља инструкције (такозвано shamt поље), док су остали бити поља резервисани, не користе се.

Поред инструкција померања; xor, xori, slt, sltu, slti и sltiu имплементиране су аналогно. ALU јединица израчунава све инструкционим скупом дозвољене операције у исто време док контролна логика врши одабир операције на основу декодоване инструкције – сигнала из контролног декодера. Детаљи о формату и операцији коју свака од инструкција обавља налазе се у [1]. immediate модул измењен је да би инструкције које користе константу рачунале са исправним вредностима.

Инструкција lui декодира се у ID фази када се екстрахује и imm поље унутар immediate блока. У EX фази, контролни сигнал set_a_to_0_i (улаз за datapath) поставља један, први операнд ALU јединице на нулу, док се други операнд поставља на излаз из immediate блока. На овај начин вредност imm поља преноси се у даље фазе проточне обраде – ALU врши сабирање imm поља са нулом. Нема уписа у меморију, док се у WB фази податак уписује у одредишни rd регистар, аналогно као и код I и R типа инструкција. immediate блок задужен је за померање вредности imm поља у лево за одговарајући број места. Инструкција auipc, такође U типа, фунционише на сличан начин, с тим што се уместо нуле као први операнд ALU јединице доводи вредност програмског бројача регистрована кроз id / ex регистар. Овим понашањем управља контролни сигнал a_is_pc_i. ALU овако врши сабирање imm поља инструкције са вредношћу програмског бројача инструкције. Унутар контролног декодера и ALU декодера додата је одговарајућа логика како би у случају auipc (и lui), из ALU био изабран / изашао резултат операције сабирања. Аналогно са I и R типом, нема уписа у меморију док се у WB фази (регистрована) вредност на излазу из ALU уписује у одредишни rd регистар.

Инструкције учитавања у регистар lb, lh, lbu, lhu имплементиране су додавањем новог, sign_extend модула унутар datapath-a. Донета је архитектонска одлука да се из меморије увек учитава цела меморијска реч, док процесор одсеца непотребне бите и извршава проширење знака (пре уписа у одредишни регистар). С тим у вези BRAM модул није значајно мењан. Исправљене су ситне грешке примећене у овом модулу током отклањања грешака процесора – на пример, један од важних сигнала није се нашао на листи осетљивости. sign_extend модул налази се унутар MEM фазе, да би у WB фази резултат читања из меморије могао да се упише у регистарску банку пре него што се изврши рачунање услова за условне скокове унутар ID фазе. Због уписа у регистар на опадајућу ивицу тактног сигнала није потребно прослеђивање из WB у ID фазу. На основу funct3 поља контролна логика генерише сигнале rd_be_i и mem_unsigned_i који контролишу који бити прочитане речи из меморије ће бити прослеђени sign_extend јединици и да ли се у WB фазу прослеђује одсечена, unsigned вредност или излаз из sign_extend блока, signed вредност (мултиплексер). Контролни сигнали лако се генеришу одсецањем funct3 поља, у ID фази, након чега се кроз проточну обраду прослеђују до MEM фазе. ALU јединица функционише исто као и у случају lw инструкције.

Подршка за sb и sh инструкције додата је тако што је унутар контролног декодера ширина data_mem_we_o излазног сигнала проширена са једног на два бита. Користе се следеће вредности: “01” за sb, “10” за sh и “11” за sw. Овај сигнал се кроз проточну обраду преноси до MEM фазе, јер се на овом месту врши упис у меморију. Мултиплексер унутар controlpath-a процесора измењен је да уместо две опције бира 3 – у зависности од тога који битови се уписују у меморију. Остатак логике исти је као онај за sw инструкцију.

Било је посебно важно обезбедити подршку за условне скокове, с обзиром на то да ове инструкције доводе до управљачких хазарда. Сви условни скокови су B типа. Архитектура факултетског процесора специфична је по томе што програмске скокове решава већ у ID фази. С тим у вези, у datapath додат је branch_eval блок, чији задатак је евалуација / рачунање услова скока у зависности од операције (операцију бира контролни сигнал branch_op_i). Исто као у случају ALU јединице све операције врше се у паралели, док контролна логика одлучује који сигнал ће постати резултат / исход скока. Контролни сигнал операције која се

извршава је funct3 поље инструкције. Контролној логици се као улаз прослеђује израчунат исход скока. Овај сигнал користи јединица за предвиђање скокова и контролна логика која контролише мултиплексер испред програмског бројача. EX, MEM и WB фазе не раде ништа – сабирач у EX фази не користи се јер се адреса скока рачуна у ID фази посебним сабирачем.

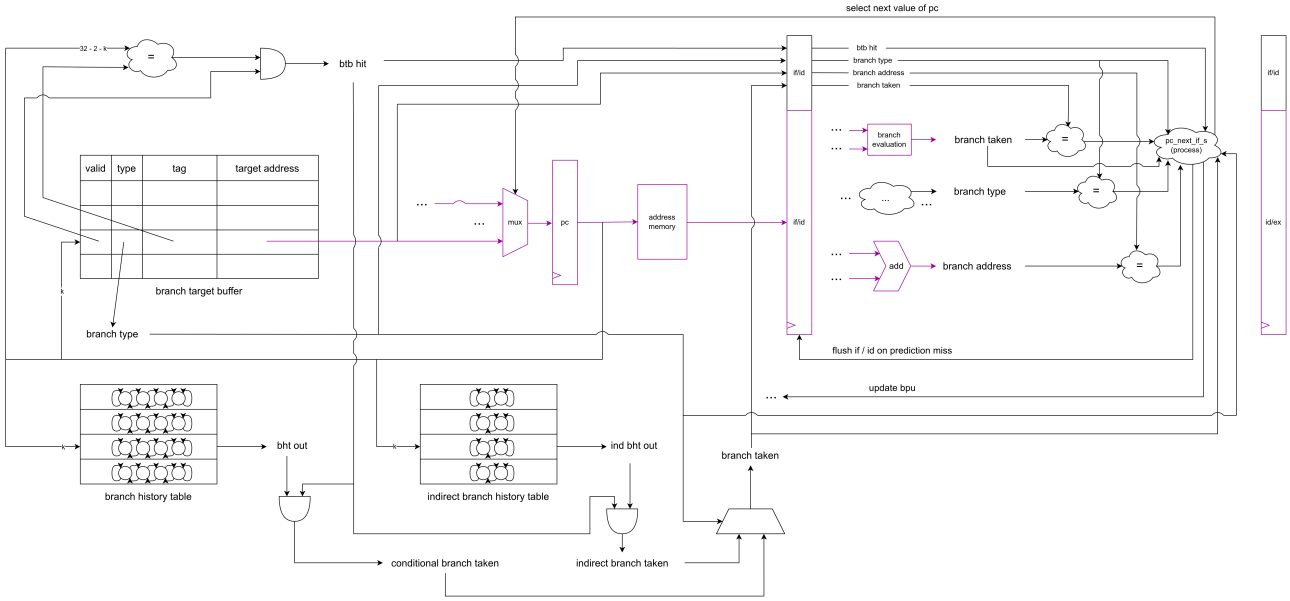
Безусловни скок jal је инструкција J типа, те је у складу са тим ажуриран immediate блок како би се на правилан начин екстраховала константа из машинског кода инструкције. Рачунање адресе скока врши се у ID фази, док се улаз сабирача мултиплексира у зависности од тога да ли се ради о jal или о jalr инструкцији. У случају jal инструкције вредност константе (поље imm) сабира се са вредношћу програмског бројача. datapath рачуна ову адресу скока док је контролна логика задужена за њено прослеђивање програмском бројачу. Такође је потребно да се повратна адреса, тј вредност pc + 4 упише у одредишни регистар у WB фази, док у MEM фази нема уписа у меморију. С тим у вези контролни декодер допуњен је логиком која подиже we сигнал регистарске банке у случају да је декодована jal инструкција. У EX фази било које инструкције безусловног скока, било да је реч о jal или jalr инструкцији, мултиплексером на излазу из ALU бира се да ли се у следећу фазу проточне обраде преноси излаз из ALU или вредност pc + 4 (регистрована). На овај начин јединица за послеђивање и даље ће радити како је првобитно планирано без икаквих измена, јер се у MEM фази налази исправна вредност која тек треба да се упише у rd регистар.

Слично jal инструкцији, jalr израчунава адресу скока у ID фази, док се у EX фази преко мултиплексера даље прослеђује вредност повратне адресе, а у WB фази врши уписивање повратне адресе у rd. Разлика је у типу инструкције (I тип) и операнду сабирача за програмске скокове унутар ID фазе. Уколико је декодована jalr инструкција операнд сабирача (у ID фази) уместо вредности програмског бројача постаје branch_condition_a_ex_s – вредност rs1 прочитана из регистарске банке или излаз сабирача из EX фазе у MEM фази уколико је дошло до прослеђивања операнда. Овај сигнал већ је постојао у почетном коду процесора. Контролна логика задужена је за прослеђивање вредности израчунате адресе скока програмском бројачу (мултиплексер испред pc-ја). Занимљиво је да по спецификацији jalr инструкција поставља најнижи бит адресе на коју се скаче на нулу. У hdl-у ово је имплементирано тако што је најнижи бит сабирача за програмске скокове постављен на нулу, што је дозвољено јер је најнижи бит било које адресе на коју се врши скок увек једнак нули.

Јединица за прослеђивање побољшана је додавањем додатних “in use” сигнала, док је логика остала мање више иста. Додати сигнали обезбеђују да се прослеђивање не деси уколико rs1 и / или rs2 поља нису коришћена у оквиру декодоване инструкције. datapath је проширен имајући у виду постојеће путање за прослеђивање, тако да су оне остале исте као што су и биле код почетног процесора.

Имплементацијом jalr инструкције долази до појаве хазарда података у систему, у истим случајевима као што је то било и код почетног процесора, за условне скокове. С тим у вези јединица за отклањање хазарда (hazard_u) проширена је на адекватан начин, тако да се у обзир узму и jalr скокови, а не само условни скокови као што је то био случај са полазном архитектуром. jalr изазива хазарде због могуће неспремности операнда rs1, док се код jal скока хазарди не јављају јер адреса скока не зависи од вредности било ког регистра, већ је кодована унутар саме инструкције (директан скок).

Јединица за предвиђање скокова



Слика 1. Архитектура јединице за предвиђање скокова и њена интеграција у постојећу процесорску архитетуру (поједностављено). Обојени део представља datapath део процесора.

Статички предиктор полазне архитектуре у потпуности је замењен предиктором заснованом на алгоритму Смита, тј бројачу са засићењем (слика 1). Коришћени су двобитни бројачи за условне скокове и једнобитни бројачи за безусловне, jalr скокове. Модул за предвиђање скокова предвиђа исход скока и адресу скока за условне скокове, док је за безусловне скокове довољно само предвиђање адресе скока. Код индиректних скокова (само jalr унутар RV32I сета инструкција) уведен је хистерезис – предвиђена адреса користи се тек након што је претходно већ једанпут била тачно предвиђена. Овакав приступ значајно повећава перформансе процесора у одређеним сценаријима [4]. Индиректан скок је скок код ког адреса скока зависи од вредности чуване у регистру процесора, а не у самој инструкцији. Код директних безусловних скокова (jal) нема потребе за хистерезисом јер је адреса скока увек иста за сваку адресу саме инструкције скока. Условни скокови такође имају хистерезис јер се користе двобитне бројаче са засићењем.

Предиктор се налази у IF и ID фази контролне логике. vhd модел састоји се од модула btb, bht и ind_bht који ће бити објашњени у наставку. Излаз из предиктора је предвиђен исход скока и предвиђена адреса скока. Њих процесор користи у IF фази, при чему уколико је скок исправно предвиђен не долази заустављања проточне обраде. У колико то није био случај if / id регистар се чисти док се програмском бројачу прослеђује одговарајућа адреса. Улази у предиктор су израчуната адреса скока и израчунат исход скока у ID фази. На основу ових повратних информација модул за предвиђање заправо “учи”.

Предвиђање адресе се врши тако што се кешира вредност адресе скока унутар специјализованог кеша који се назива btb (eng. *branch target buffer*). Дизајниран је direct-mapped кеш, ради једноставности. btb подржава истовремено читање и писање. Последњих k бита програмског бројача користе се за адресирање овога кеша, док се у самом кешу чувају тагови (битови програмског бројача виши од k) и циљна адреса скока придружена том тагу. Изабрано је да k буде једнако 5, што значи да адресни кеш (btb) има 32 уноса, што је сасвим довољно за мале тест програме. Треба напоменути да последња два бита програмског бројача нису коришћена за адресирање кеша јер су увек 0 те би конфликти у кешу били чешћи уколико би се изабрао овај приступ. btb између осталог уз сваки унос складишти и бит валидности података као и бит за метаподатке – говоре о ком типу скока се ради. Исто као и

код регистарске банке, читање је асинхроно док је упис синхрон и врши се на опадајућу ивицу тактног сигнала.

Предвиђање исхода скока врши се помоћу структуре која се назива bht (eng. *branch history table*). Структура садржи 2^k бројача са сатурацијом, а ширину бројача могуће је подесити преко параметра CNT_WIDTH. Сваки од бројача понаша се исто – уколико је дошло до поготка при предвиђању, бројач се увећава, а уколико је дошло до промашаја смањује се. Вредност бројача никада не прелази испод нуле нити изнад максималне вредности, због чега се и назива “са сатурацијом”. Највиши бит бројача представља предвиђену вредност програмског скока. Табела се адресира на исти начин као и btb – са k бита програмског бројача. За условне скокове користе се двобитни бројачи (најбоље су се показали у пракси), док се за jalr скокове користе једнобитни бројачи за имплементацију хистерезиса.

Погодак у кешу дешава се када је кеширани таг једнак са тагом тренутне вредности програмског бројача и уколико је бит валидности подигнут. Предвиђа се да је скок извршен уколико је дошло до поготка у адресном btb кешу и bht излаз је висок. Ако један од ових услова није задовољен предвиђа се да скок није извршен. У случају индиректног скока нема предвиђања искода скока већ се овај излаз из предиктора користи на другачији начин. У овом случају bht уноси представљају историју погодака адресе за jalr инструкцију. Уколико је адреса скока први пут тачно предвиђена процесор сматра да је предвиђена адреса скока нетачна. Тек уколико је адреса скока тачно предвиђена други пут за редом долази до скока у IF фази. Безусловни директни скокови (jal) користе само кеш адреса, није потребан хистерезис нити предвиђање скока, те никада нема уписа у bht. Јединица за предвиђање програмских скокова садржи две bht меморијске структуре, једну са двобитним бројачима са сатурацијом за условне скокове и другу са једнобитним бројачима са сатурацијом за индиректне безусловне скокове. На основу метаподатака прочитаних из btb адресног кеша предиктор “зна” о којој врсти скока је реч.

Контролни сигнал pc_next_sel_i унутар datapath-а процесора проширен је са једног на два бита да би могле да се покрију све вредности које програмски бројач може добити у следећем циклусу. Уколико нема програмског скока програмски бројач добија вредност pc + 4. Уколико је предвиђање било успешно предвиђена адреса уписује се у pc. Када се предвиђање исхода скока испостави као нетачно и скок се десио прослеђује се израчуната вредност адресе скока, док се у случају када је предвиђени исход био нетачан и скок се није десио прослеђује адреса инструкције након иструкције погрешно предвиђеног скока.

Битови за метаподатке складиштени у адресном кешу служе за идентификацију типа инструкције гранања током извођења предвиђања у IF фази. Потребно је два бита за метаподатке из разлога што се предвиђање условних директних, безусловних директних и безусловних индиректних скокова врши на другачији начин (3 дозвољене вредности поља за метаподатке). Ово поље је неопходно из разлога што у IF фази и даље није познато о ком типу скока се ради, јер инструкција тек треба да се декодује. Контролна логика бира одговарајући улаз у мултиплексер испред програмског бројача на основу сигнала из јединице за предвиђање. Једну фазу проточне обраде касније, у ID фази, контролна логика врши flush операцију уколико скок није правилно предвиђен. Пореде се резултати предвиђања из IF фазе (исход скока, адреса скока и метаподаци скока) са израчунатим, стварним вредностима из ID фазе. Предвиђање је успешно уколико дође до потпуног поклапања. У овом случају је довољно ажурирати само bht структуру. btb се не ажурира јер су у случају исправно предвиђеног скока подаци унутар њега сигурно валидни. Због тога што унутар јединице за предвиђање скокова постоје две bht структуре, свакој од њих додељен је посебан we улаз. en улаз у bpu (eng. *branch prediction unit*) служи да онемогући било какав упис у структуре које се налазе унутар bpu у случају да је дошло до заустављања проточне обраде процесора услед хазарда података.

Верификација и тест

Искоришћено је постојеће верификационо окружење факултетског процесора. reset сигнал подигнут је након 200 ms, јер је приликом развоја уочено да се постојућим верификационим окружењем не могу верификовати програми који садрже само једну линију машинског кода. Меморија за податке није интензивније коришћена приликом верификације, осим у случају провере инструкција за рад са меморијом. Направљена је wcfg датотека да би се у симулацији могли видети сви унутрашњи сигнали процесора подељени по хијерархији на којој се налазе. Овај приступ умногоме је олакшао процес верификације. wcfg датотека аутоматски је додата у пројекат. Тест програми коришћени за верификацију процесора писани су у асемблерском језику, псеудо-инструкцијама које су потом претворене у машински код помоћу симулатора Ripes. Тест програми написани су тако да се вредности одговарајућих регистара у регистарској банци на крају извршавања ажурирају уколико је инструкција успешно извршена. Покретањем програма у симулатору Ripes могуће је утврдити на који начин исправан систем треба да се понаша, исход програма. Занимљиво је што је у току развоја процесора нађена грешка у начину на који Ripes симулатор обрађује инструкцију jalr, проблем који је пријављен развојном тиму и убрзо након исправљен у новој верзији симулатора [3]. Тест програми написани у асемблерском језику налазе се у директоријуму RISCV_tb/test_source_codes, док се исти програми претворени у машински код налазе у директоријуму RISCV_tb/test_machine_codes.

Што се тиче верификације блока за предвиђање програмских скокова, у симулацији је потврђено да програм ради исправно упркос томе што се сигнал if_id_flush_s не активира (нема застоја проточне обраде при програмском скоку). Предиктор је тестиран покретањем три примера од којих сваки тестира једну врсту програмског скока. Први пример simple_loop је празна петља која се понавља 10 пута. Пример тестира функционалност предиктора у случају условних скокова. У симулацији је утврђено да предиктор погреши прва два пута као и последњи пут, што је очекивано понашање, с обзиром на то да су све вредности унутар bht “00” након ресета (strongly not taken). Други пример, bp_jal_example је бесконачна безусловна петља која у сваком циклусу повећава вредност регистра x10 за 1. У симулацији се да приметити да процесор само једном извршава flush if / id регистра, први пут када наиђе на инструкцију скока, јер се у том тренутку у времену адреса скока инструкције још увек не налази у адресном кешу. У сврхе теста jalr инструкције написан је сличан тест програм, bp_jalr_example – бесконачна петља која у сваком циклусу повећава вредност x11 регистра за 1 (прецизније, структура програма сличнија је do-while петљи). У симулацији је проверено да хистерезис постоји, адреса скока кеширана је први пут када процесор дође до инструкције скока и предвиђање се води као нетачно, други пут такође не долази до скока у IF фази и предвиђање је и овај пут нетачно због тога што постоји хистерезис (без обзира на то што је адреса скока исправна и кеширана). Коначно, трећи пут долази до скока већ у IF фази.

С обзиром на то да фокус пројекта није био на верификацији процесора већ на имплементацији додатних функционалности, може се претпоставити да и даље постоје скривене грешке у дизајну, с обзиром на то да је процесор тестиран на ограниченом броју тест програма. Сви тест програми унутар репозиторијума се проверено исправно понашају приликом покретања на процесору, чиме се поједностављена верификација (eng. *sanity check verification*) може сматрати завршеном.

Закључак

Покривено је једноставно решење ублажавања негативног деловања управљачких хазарда на брзину извршавања програма на bare-metal RV32I архитектури процесора. Нису имплементиране инструкције из целокупног RV32I скупа инструкција, што би могао да буде један начин да се пројекат додатно прошири. Такође, jalr инструкција и инструкције условног гранања деле исти адресни кеш (btb) иако поседују одвојене bht структуре, што може представљати уско грло при покретању већих програма, како стопа конфликтних промашаја у адресном кешу расте. Није имплементиран RAS (eng. *Return Address Stack*), те се може претпоставити да перформасе јединице за предвиђање скокова у случају предвиђања безусловних скокова нису на довољно високом нивоу уколико постоји доста позива функција / процедура унутар програма. Адресни кеш могуће је проширити да буде два или три сет-асоцијативни како би се стопа конфликтних промашаја смањила за велике програме. Није улажено у превише детаља предвиђања безусловних скокова, фокус је био на предвиђању условних скокова и делимично предвиђању безусловних индиректних скокова. Обе теме могуће је проширити у будућем раду, на начин такав да предиктор даје већу стопу погодака у општем случају. Уз мале измене у тест окружењу процесора чак је могуће и покренути једноставан benchmark програм у сврхе мерења ефикасности имплементираног branch predict решења, с обзиром на то да је процесор у стању да покрене произвољни програм, под условом да се не користе привилеговане инструкције. Да би се измерile перформансе предиктора (проценат погодака, промашаја) могуће је изменити верификационо окружење тако да се броје циклуси активног извршавања програма. Још један од потенцијалних начина на који би се верификационо окружење могло додатно унапредити је аутоматизација верификације инструкционог сета процесора у корист мануелне анализе мноштва сигнала од стране инжењера при покретању симулације.

Извори

[1] S. L. Harris and D. Harris, Digital design and computer architecture: RISC-V edition. Morgan Kaufmann, 2022.
[2] M. B. Petersen, Ripes, https://github.com/mortbopet/ripes, GitHub, 2026.
[3] lazarmincic, Issue #477, Issue with implementation of JALR instruction, https://github.com/mortbopet/Ripes/issues/477, GitHub, 19.10.2026.
[4] K. Driesen, U. Hölzle, "Accurate indirect branch prediction," Proceedings of the 25th Annual International Symposium on Computer Architecture (ISCA '98), Barcelona, Spain, June 1998, pp. 167–178.